

Pre-class Preparation Guide

Knowledge Graph and Retrieval-Augmented Generation (RAG)

This guide will help you prepare for the tutorial before class.

The tutorial is divided into two parts:

- **Part 1 — Knowledge Graph:** Build and query a knowledge graph using Neo4j.
- **Part 2 — RAG:** Set up a Python environment and run a Retrieval-Augmented Generation (RAG) pipeline using Jupyter Notebook.

Please complete the preparation steps before class. You do **not** need to fully understand every piece of code in advance. The goal is to make sure that your environment and basic accounts are ready so that we can focus on the concepts and hands-on activities during the tutorial.

Part 1 — Knowledge Graph

0. Download the Dataset

Download the tutorial materials from the course GitHub repository:

[Course GitHub Repository](#)

Open the folder:

```
01_tutorial_KG
```

The folder contains two datasets:

```
01_tutorial_KG
├── dataset01_in-class-exercise
│   └── Data used for the in-class exercise
└── dataset02_tutorial_ifc2graph
    └── Data used for the tutorial exercise
```

Before class, please briefly look through the tables in these two folders and try to understand what information each table contains.

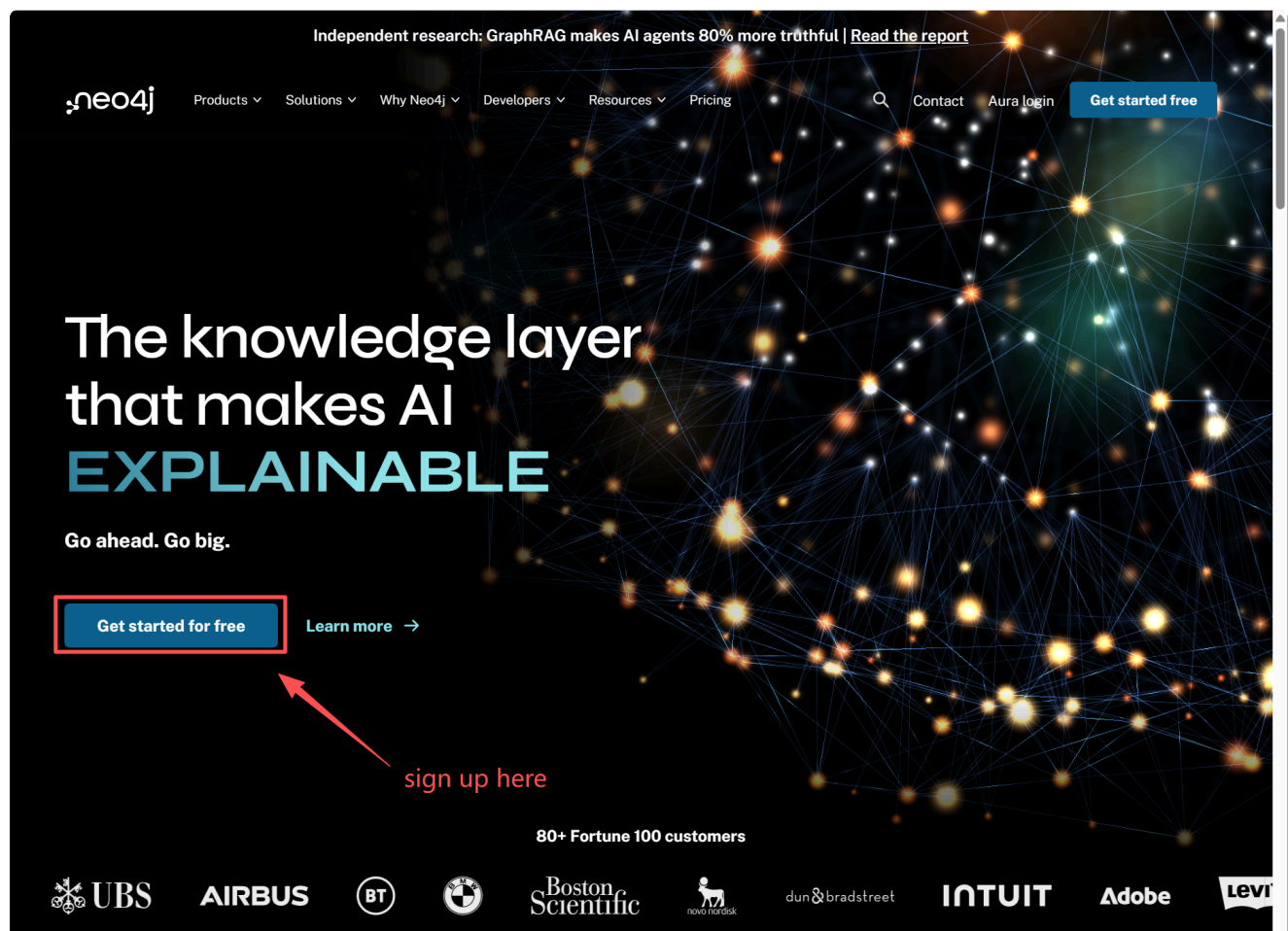
You do **not** need to build the knowledge graph before class. We will build it together during the tutorial.

1. Create a Neo4j Account

Go to:

[Neo4j](#)

Create a Neo4j account and log in.



2. Create a Neo4j Instance

After logging in, enter the **Neo4j Aura** management interface.

From the sidebar:

1. Select **Instance**.
2. Click **Create instance**.
3. Select the **Free** instance.

The Free instance is sufficient for this tutorial.

The screenshot shows the Neo4j Cloud console interface. On the left is a sidebar with navigation links: Get started, Developer hub, Data services (Instances, Graph Analytics, GraphQL Data APIs), Studio (Agents, Import, Query, Bloom, Dashboards, Document Intelligence), Operations, Project, and Learning. The main area is titled 'Create instance' and includes a confirmation message: 'Confirm your tier. We'll recommend default values to help you get going. [See full feature comparison](#)'. Three tiers are presented: Professional (\$0.09 GB/hour), Business Critical (\$0.20 GB/hour), and Free (\$0 GB/hour). The Free tier is selected with a radio button and highlighted by a red arrow. Below the tiers, there's a link to 'See Virtual Dedicated Cloud' and an 'Instance details' section with a text input field containing 'Instance01'.

Tier	Price	Features
Professional	\$0.09 GB/hour	Database monitoring with logs and metrics, Predefined RBAC, Backups, 7d retention, Graph algorithms, vector optimized, Standard support
Business Critical	\$0.20 GB/hour (min 2GB)	Advanced monitoring, 3rd party integrations, Custom RBAC, IP filtering, SSO, Backups, 30d retention, Graph algorithms, vector optimized, 24/7 support, 99.95% uptime SLA, Multiple databases (Preview)
Free	\$0 GB/hour	Up to 200k nodes and 400k relationships, Limited memory and vCPU, Limited backups, Auto-deleted after 30 days of inactivity

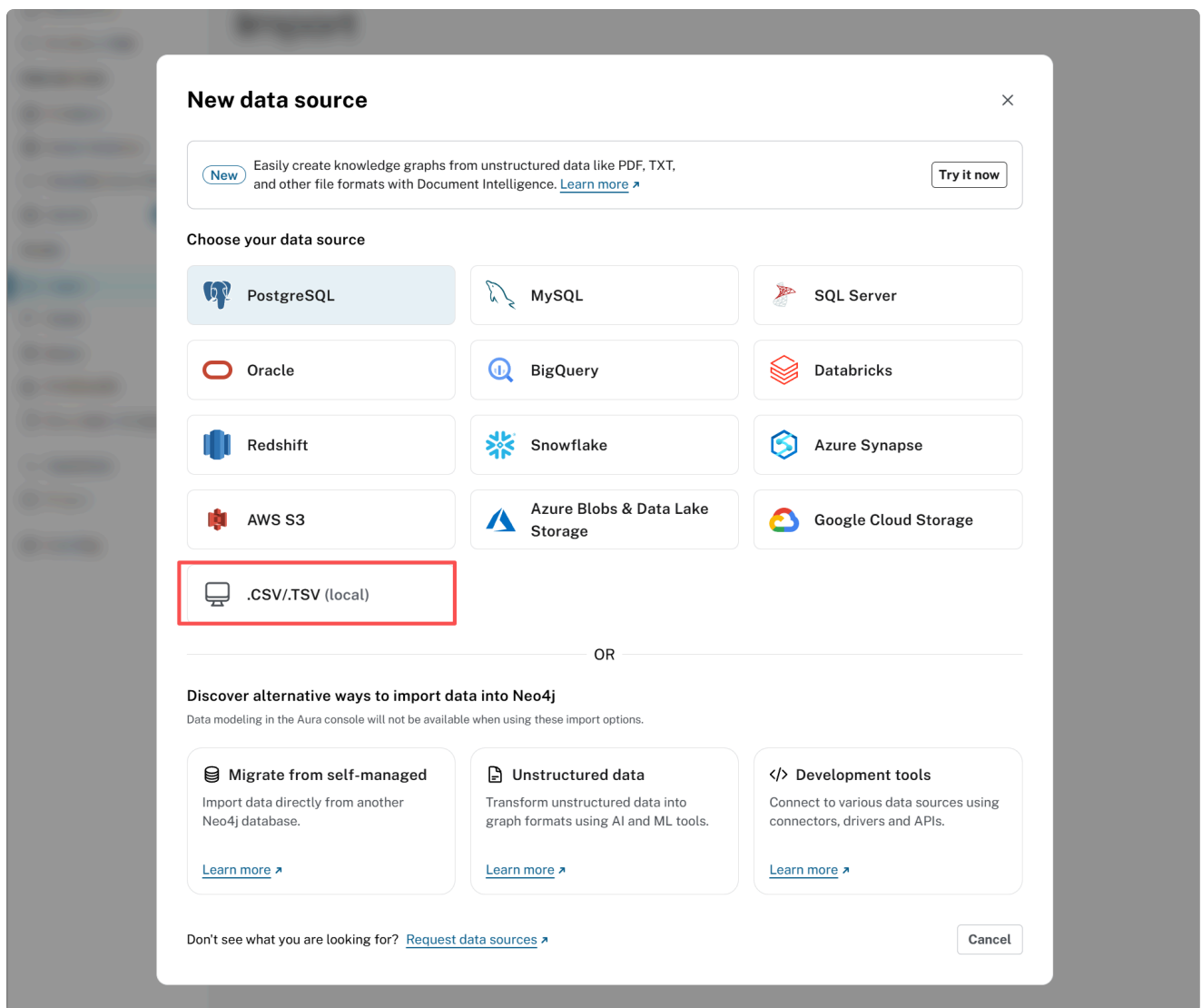
3. Import the Dataset

After creating your instance:

1. Select **Import** from the sidebar.
2. Click **New data source**.
3. Select **.csv/.tsv**.
4. Upload the CSV files from:

```
dataset01_in-class-exercise
```

You can use this dataset to become familiar with the Neo4j data import workflow before we start building the knowledge graph.



4. Build a Knowledge Graph

In the **Data Sources** section, you can see the first few rows of the imported data.

For this tutorial, we will use **Define manually** to construct the graph.

4.1 Define a Node

We normally start by defining the nodes.

A node requires a **Label**. A label represents the type or category of a node.

For example, in our dataset:

User

is a type of node.

To define the `User` node:

1. Enter `User` as the **Label**.

2. Select:

```
stackoverflow.nodes.User.csv
```

as the data source.

3. Use **Map from table** to import the columns as node properties.

4. Define a unique **ID** for the node.

The ID works like an identification number: **each ID should correspond to one unique record**.

For example:

```
ID = 12345  
Name = John
```

The name may be shared by multiple users, but the ID should uniquely identify a user.

neo4jaura / Shanghai Jiao Tong University / New project

Trial - 10 days left Feedback

Import models / Untitled model 3

Generate model

Get started

Developer hub

Data services

Instances

Graph Analytics

GraphQL Data APIs

Agents **New**

Studio

Import

Query

Bloom

Dashboards

Document Intelligence

Operations

Project

Learning

Data source **Local** Browse

stackoverflow.node...
answer_id 2155
link https://sta...
title Return &#...
is_accept... true
body_mar... The error i...
score 1
uuid 69272967

stackoverflow.node...
comment... 5943
score 0
uuid 122336972
link https://sta...

stackoverflow.node...
question_id 0
link https://sta...
accepted... 65954158
creation_... 16105257...
view_count 70
answer_c... 1
body_mar... I want to c...
uuid 65697972
title Create gr...

stackoverflow.node...
tag_id 1679
name neo4j
link https://sta...

stackoverflow.node...
user_id 604

Select your model starting point

Define manually
Start with an empty canvas and sketch your model and data mapping manually

Generate from schema
Define a graph model and data mapping automatically from tables and constraints

Generate with AI
Define a graph model and data mapping automatically using AI on data source metadata
Always review the accuracy of AI generated output

4.2 Define a Relationship

Next, we can connect two nodes with a relationship.

Drag from the blue edge of the `User` node to create another node. In our example, we can create a:

Question

node.

We then define the relationship between them.

From the dataset, we know that a user can **ask** a question:

User → ASKED → Question

Therefore, we can use:

```
stackoverflow.relationships.ASKED.csv
```

as the data source for the relationship.

For this tutorial, defining properties and IDs for relationships is optional, so we will not cover them in detail.

The screenshot shows the Neo4j Studio interface. On the left, a sidebar contains navigation options like 'Get started', 'Developer hub', 'Data services', and 'Studio'. The 'Import' option is highlighted. The main workspace displays a graph with two nodes, 'User' and 'Question', connected by a relationship labeled 'ASKED'. The 'User' node has a green checkmark, and the 'Question' node has a green checkmark. The relationship is represented by a double-headed arrow with the label 'ASKED' in the middle. On the right, a panel titled 'Relationship type' shows 'ASKED' as the selected type. Below this, there are tabs for 'Properties', 'Constraints', and 'Indexes'. The 'Properties' tab is active, showing a dropdown menu for 'stackoverflow.relationship...' and a 'Map from table' button. Below the properties, there is a 'Node ID mapping' section with a table showing the mapping between nodes and their ID properties. The table has columns for 'Node', 'ID property', and 'ID column'. The 'From' row shows 'User' mapped to 'user_id' in the 'user_id' column. The 'To' row shows 'Question' mapped to 'question_id' in the 'question_id' column. At the bottom of the right panel, there is an 'ID' section with a dropdown menu for selecting an ID property.

Node	ID property	ID column
From: User	user_id	user_id
To: Question	question_id	question_id

At this point, we have created a basic graph:

```
User —ASKED—> Question
```

You can continue to add other nodes and relationships following the same process.

Can AI help build the graph?

Yes. If the structure of your data is clear, AI can help you generate a graph model.

The dataset used in this tutorial is relatively well structured, so AI-generated graph definitions can be useful.

However, **please be careful when using AI with your own datasets**.

AI may misunderstand the meaning of columns, relationships, or IDs. Always check the generated graph model against the original data before importing it.

The screenshot displays the Neo4j Studio interface for 'Untitled model 4'. The left sidebar shows the 'Import' tab selected under the 'Studio' section. The main workspace shows a graph model with the following structure:

- Nodes:** Tag, Question, Comment, User, Answer.
- Relationships:**
 - TAGGED (Tag to Question)
 - COMMENTED ON (Question to Comment)
 - ASKED (User to Question)
 - PROVIDED (User to Answer)
 - ANSWERED (Answer to Question)

The right-hand panel shows the 'Properties' tab for the 'Question' node. It lists the following properties and their corresponding database columns:

Name	Type	Column
accep...	integer	accepted_...
answe...	integer	answer_co...
body...	string	body_mark...
creati...	integer	creation_d...
link	string	link
questi...	integer	question_id
title	string	title
uuid	integer	uuid
viewC...	integer	view_count

The ID field is set to 'questionId'.

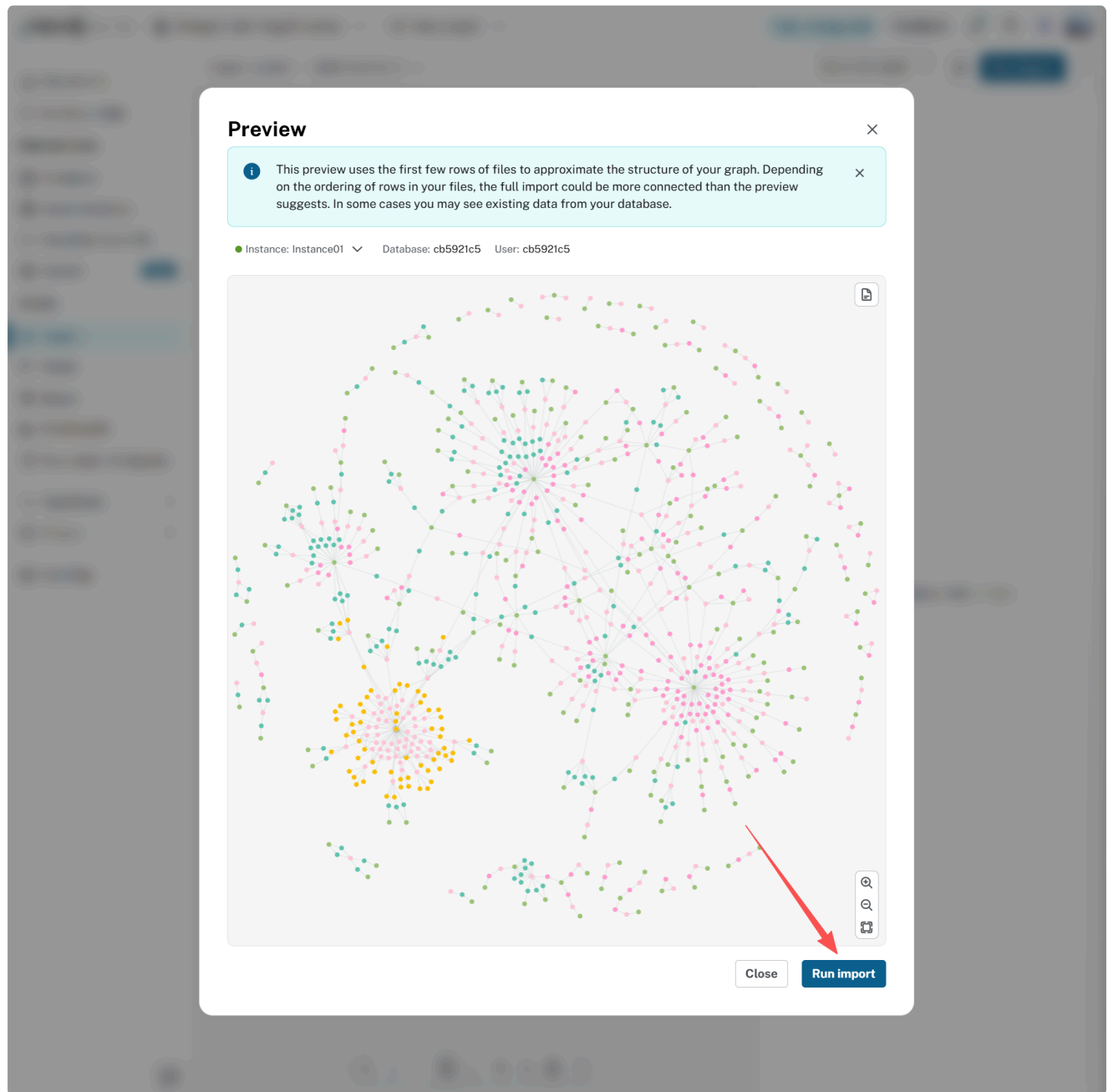
4.3 Preview and Import the Graph

When you have finished defining the graph model:

1. Click **Preview** in the upper-right corner.
2. Check whether the graph structure looks correct.
3. Click **Run import**.

The graph model will then be imported into your Neo4j instance.

Congratulations! You have created your first graph database.



5. Query the Knowledge Graph

Now we can use the knowledge graph to retrieve information.

From the sidebar, select **Query** and connect to the instance you just created.

The **Database Information** panel shows the nodes and relationships currently available in your database.

5.1 Query Using the GUI

Neo4j queries are normally written using **Cypher**, Neo4j's graph query language.

However, you do not need to know Cypher to perform simple queries.

For example, you can directly click on a node or relationship in the interface to view the corresponding data.

By default, Neo4j displays the first **25 records**.

You can change the `LIMIT` value or remove it if you want to retrieve more records.

The screenshot displays the Neo4j Aura web interface. On the left is a sidebar with navigation options like 'Get started', 'Developer hub', 'Data services', and 'Studio'. The main panel is divided into three sections:

- Database information:** Shows 6,193 nodes and 11,540 relationships. Nodes are categorized by type: Answer, Comment (highlighted with a red box), Question, Tag, and User. Relationships include ANSWERED, ASKED, COMMENTED, COMMENTED_ON, PROVIDED, and TAGGED. Property keys listed include acceptedAnswerId, answerCount, answerId, bodyMarkdown, commentId, creationDate, displayName, isAccepted, link, name, questionId, score, tagId, title, userId, uuid, and viewCount.
- Query:** A Cypher query is entered: `cb5921c5$ MATCH (n:Comment) RETURN n LIMIT 25;`. The query is executed, resulting in a graph visualization of 25 comment nodes.
- Stream:** A 'welcome' message is displayed.

At the bottom, there are two cards: 'GUIDE' titled 'Query fundamentals' and 'DATASET' titled 'Try Neo4j with the Movie Graph'.

5.2 Use AI to Generate Cypher

For more complicated queries, AI can help you generate Cypher statements.

Show me a visual representation of the graph data model.

AI may generate:

```
CALL db.schema.visualization()
```

You can provide the question to the AI assistant and run the generated Cypher query.

You can also use other AI tools, such as ChatGPT, to help you understand what the generated Cypher statement is doing.

The screenshot displays the Neo4j Aura web interface. The top navigation bar includes the Neo4j logo, project name 'Shanghai Jiao Tong University', a 'New project' button, and a 'Trial - 10 days left' badge. The left sidebar contains navigation links for 'Get started', 'Developer hub', 'Data services', 'Instances', 'Graph Analytics', 'GraphQL Data APIs', 'Agents', 'Studio', 'Import', 'Query' (highlighted), 'Bloom', 'Dashboards', 'Document Intelligence', 'Operations', 'Project', and 'Learning'.

The main content area is divided into three sections:

- Database information:** Shows 6,193 nodes and 11,540 relationships. Nodes include Answer, Comment, Question, Tag, and User. Relationships include ANSWERED, ASKED, COMMENTED, COMMENTED_ON, PROVIDED, and TAGGED. Property keys are listed for various nodes, such as acceptedAnswerId, answerCount, answerId, bodyMarkdown, commentId, creationDate, displayName, isAccepted, link, name, questionId, score, tagId, title, userId, uuid, and viewCount.
- Query 1:** The Cypher query is `cb5921c5$ CALL db.schema.visualization()`. The result is a graph visualization showing a central 'Question' node connected to 'Tag', 'Answer', and 'User' nodes. 'User' is also connected to 'Comment' and 'Answer'.
- Query 2:** The Cypher query is `cb5921c5$ MATCH (n:Comment) RETURN n LIMIT 25;`. The result is a list of 25 'Comment' nodes, represented as green dots.

After completing these steps, you should have a basic understanding of how to:

- import data into Neo4j;
- define nodes and relationships;
- build a knowledge graph;
- query a knowledge graph using the GUI; and
- use AI to assist with Cypher queries.

Congratulations! You are ready for the Knowledge Graph tutorial.

Part 2 — Retrieval-Augmented Generation (RAG)

0. Download the Tutorial Files

Download the tutorial materials from the same GitHub repository:

[Course GitHub Repository](#)

Open the folder:

```
02_RAG_tutorial
```

The folder contains:

```
RAG_Tutorial/  
├─ RAG_In_Class_Exercise.ipynb  
├─ RAG_Tutorial_Full_Pipeline.ipynb  
├─ data/  
├─ requirements.txt  
└─ .env
```

The two Jupyter notebooks are the main teaching materials:

- `RAG_In_Class_Exercise.ipynb` — materials for the in-class exercise
- `RAG_Tutorial_Full_Pipeline.ipynb` — a complete RAG pipeline for the tutorial

The `data` folder contains the documents used in the exercises.

1. Create an Account and Obtain an API Key

For this tutorial, we will use the **Alibaba Cloud Model Studio / Bailian** platform to access large language models and embedding models.

Go to:

[Alibaba Cloud Model Studio / Bailian](#)

Register an account and obtain an **API Key**.

阿里云百炼

模型 Agent TokenPlan 文档 CLI

阿里云 登录

探索模型，开始构建

复制下面内容给您的 Agent 安装 CLI

请阅读: <https://bailian.aliyun.com/cli/...>

输入想问内容...

2. Click here to obtain API Key

API Key 创建和管理 API-KEY

Token Plan 购买和管理服务

免费额度 查看和领取模型免费额度

应用 开发及管理应用

API管理

API API Key

KMS管理

API Key

1. API Key 加密存储: 新建 API Key 仅在创建后显示一次, 供用户复制或下载保存, 平台采用不可逆的加密存储技术, 用户后续无法在控制台中再次查看明文。现有 API Key 仍然可以正常使用, 功能不受影响。

2. 业务空间专属域名: 业务空间现已支持独立推理 API 域名, 具备更高并发与网络隔离能力。DashScope 域名 (dashscope.aliyuncs.com) 将于 2026年9月30日 起进入维护状态、不再支持新特性 (仍可正常使用)。建议新业务尽快迁移。 [查看迁移文档](#)

按量付费

按量付费 Base URL

copy base URL here

[https://llm-bjb74etky7j3nuou.cn-beijing.maas.aliyuncs.com/...](https://llm-bjb74etky7j3nuou.cn-beijing.maas.aliyuncs.com/)

ID	API Key	描述	归属账号	操作
3330779	sk-a****4112	LLM	1103770	查看费用 重置 编辑 更多

your API key

创建API-KEY

Please keep your API Key private. Do not share it publicly or upload it to GitHub.

2. Configure the .env File

Open the .env file in the 02_RAG_tutorial folder.

You can right-click the file and open it with Notepad. If you already have an IDE, such as VS Code, you can use your IDE instead.

Enter your API information in the corresponding fields:

```
DASHSCOPE_API_KEY=your_api_key
DASHSCOPE_BASE_URL=your_base_url
```

Replace the example values with your own API Key and Base URL.

For the chat completion model and embedding model, you can choose models from the Alibaba Cloud Model Studio documentation:

[Model Studio Model Documentation](#)

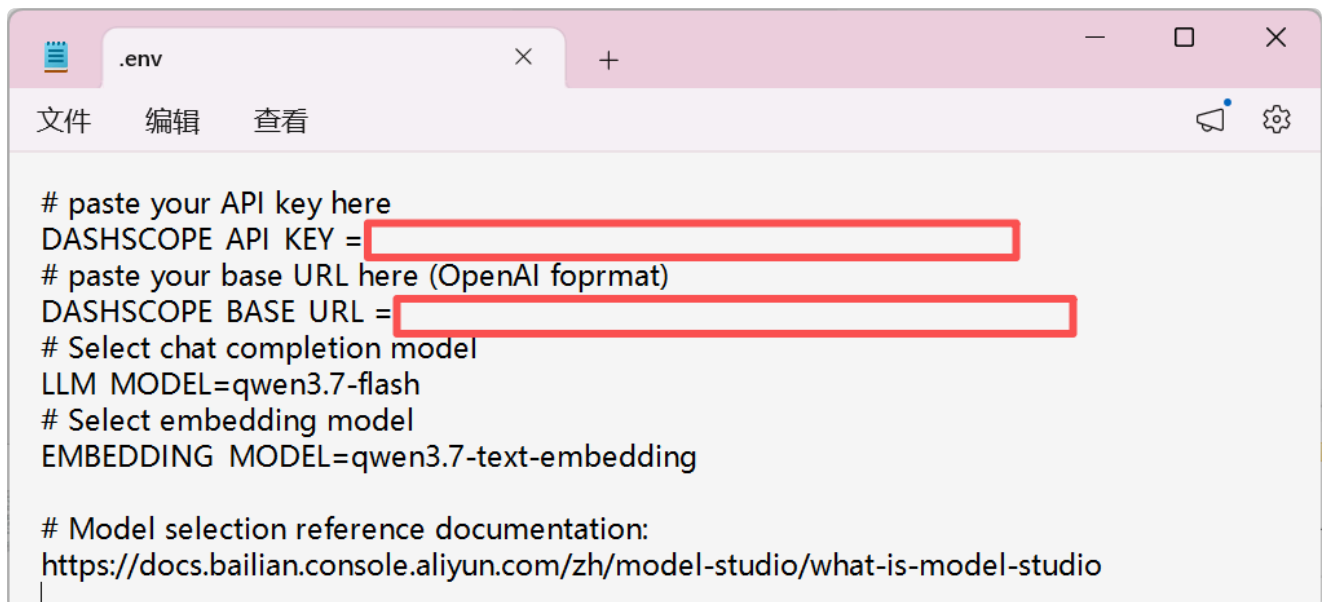
For this tutorial, the default models are:

```
Chat model:
qwen3.7-flash

Embedding model:
qwen3.7-text-embedding
```

After editing the file:

1. Press **Ctrl + S** to save.
2. Close the file.

A screenshot of a code editor window titled ".env". The editor shows the following content:

```
# paste your API key here
DASHSCOPE API KEY = 
# paste your base URL here (OpenAI foprmat)
DASHSCOPE BASE URL = 
# Select chat completion model
LLM MODEL=qwen3.7-flash
# Select embedding model
EMBEDDING MODEL=qwen3.7-text-embedding

# Model selection reference documentation:
https://docs.bailian.console.aliyun.com/zh/model-studio/what-is-model-studio
```

Red rectangular boxes highlight the empty input fields for the API key and base URL.

Important: Do not upload your `.env` file containing a real API Key to a public repository.

3. Create a Python Environment with Conda

We recommend using **Conda** to manage the Python environment for this tutorial.

3.1 Install Miniconda

Download Miniconda for Windows from:

[Miniconda — Windows Installation Guide](#)

After installation, search for:

Anaconda Prompt

in the Windows Start menu and open it.

3.2 Create a Virtual Environment

Run:

```
conda create -n myenv python=3.10
```

Here, `myenv` is the name of your virtual environment.

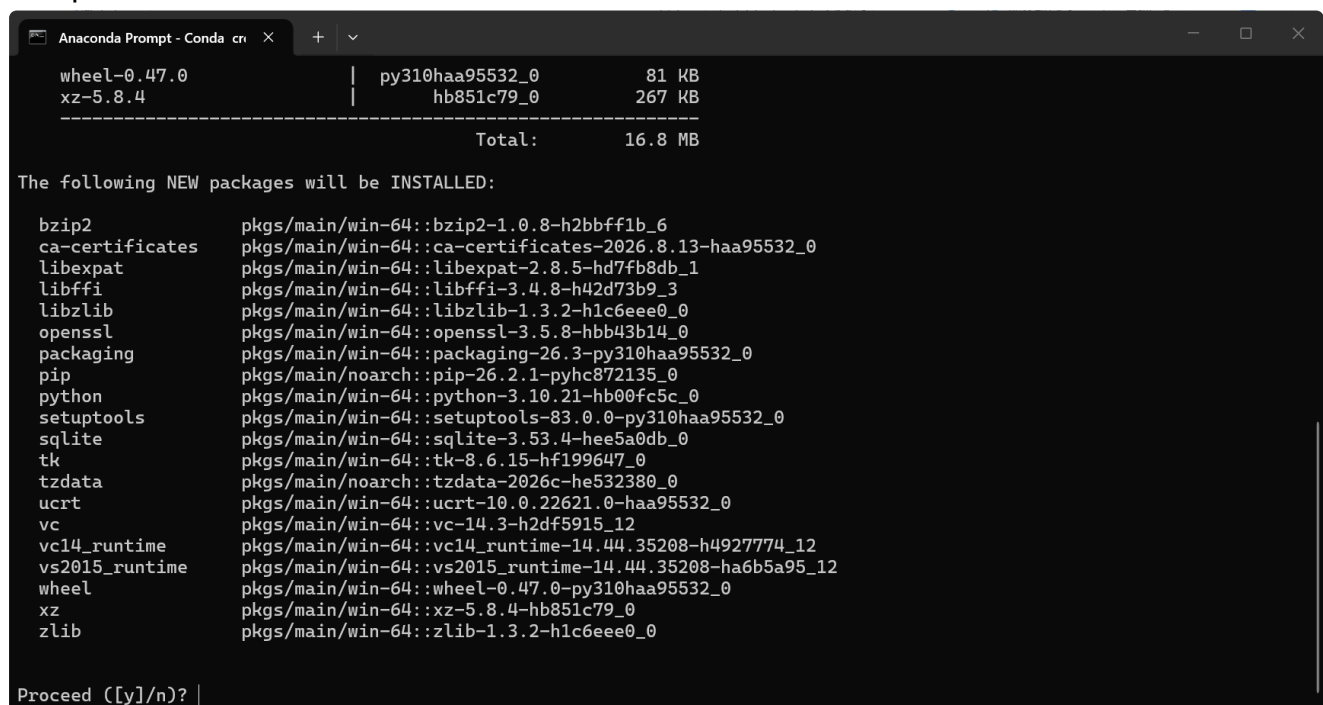
You can replace `myenv` with another name. For example:

```
conda create -n tutorial python=3.10
```

When Conda asks you to confirm the installation, enter:

y

and press Enter.



```
Anaconda Prompt - Conda cri X + v
wheel-0.47.0      | py310haa95532_0      81 KB
xz-5.8.4          | hb851c79_0           267 KB
-----
Total:           16.8 MB

The following NEW packages will be INSTALLED:

bzip2                pkgs/main/win-64::bzip2-1.0.8-h2bbff1b_6
ca-certificates      pkgs/main/win-64::ca-certificates-2026.8.13-haa95532_0
libexpat             pkgs/main/win-64::libexpat-2.8.5-hd7fb8db_1
libffi               pkgs/main/win-64::libffi-3.4.8-h42d73b9_3
libzlib              pkgs/main/win-64::libzlib-1.3.2-h1c6eee0_0
openssl              pkgs/main/win-64::openssl-3.5.8-hbb43b14_0
packaging            pkgs/main/win-64::packaging-26.3-py310haa95532_0
pip                  pkgs/main/noarch::pip-26.2.1-pyh872135_0
python               pkgs/main/win-64::python-3.10.21-hb00fc5c_0
setuptools           pkgs/main/win-64::setuptools-83.0.0-py310haa95532_0
sqlite               pkgs/main/win-64::sqlite-3.53.4-hee5a0db_0
tk                   pkgs/main/win-64::tk-8.6.15-hf199647_0
tzdata               pkgs/main/noarch::tzdata-2026c-he532380_0
ucrt                  pkgs/main/win-64::ucrt-10.0.22621.0-haa95532_0
vc                    pkgs/main/win-64::vc-14.3-h2df5915_12
vc14_runtime         pkgs/main/win-64::vc14_runtime-14.44.35208-h4927774_12
vs2015_runtime       pkgs/main/win-64::vs2015_runtime-14.44.35208-ha6b5a95_12
wheel                pkgs/main/win-64::wheel-0.47.0-py310haa95532_0
xz                    pkgs/main/win-64::xz-5.8.4-hb851c79_0
zlib                 pkgs/main/win-64::zlib-1.3.2-h1c6eee0_0

Proceed ([y]/n)? |
```

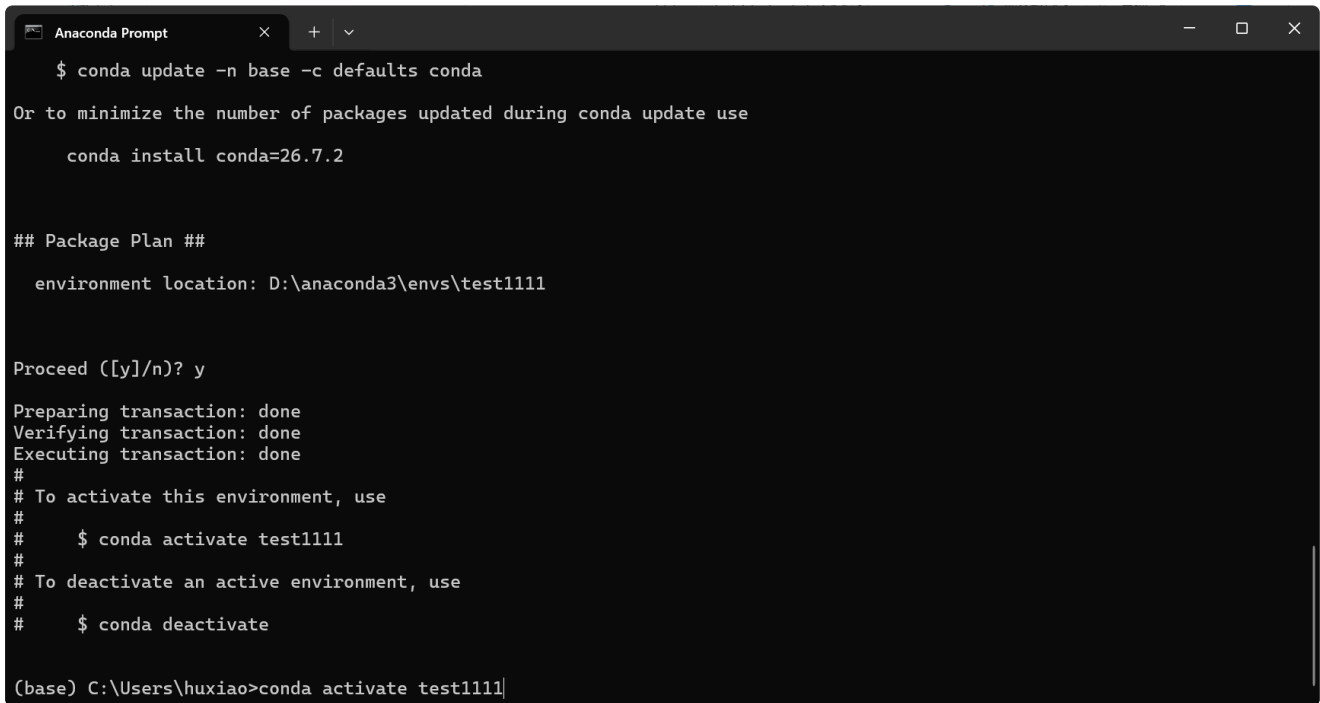
After the environment has been created, activate it:

```
conda activate myenv
```

You should now see something similar to:

```
(myenv) C:\Users\%USERNAME>
```

at the beginning of your command line.

A screenshot of the Anaconda Prompt window. The terminal shows the command '\$ conda update -n base -c defaults conda' and its output, including a package plan for environment 'test1111'. It prompts to proceed with the update, which is confirmed with 'y'. The output shows the transaction is prepared, verified, and executed successfully. It then provides instructions on how to activate and deactivate the environment. Finally, the user enters the command '(base) C:\Users\huxiao>conda activate test1111'.

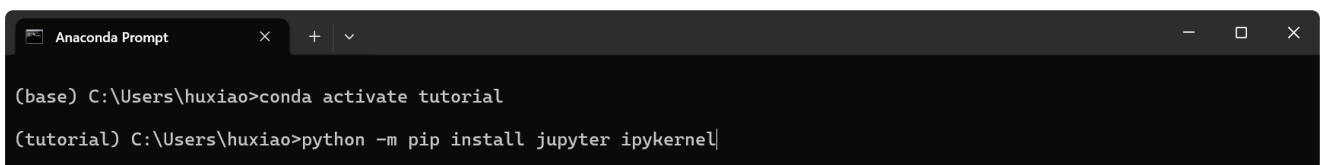
3.3 Install Jupyter Notebook

With your virtual environment activated, run:

```
python -m pip install jupyter ipykernel
```

This installs Jupyter Notebook and the Python kernel required to run the tutorial.

If you already have Jupyter Notebook installed and know how to use it, you can skip this step.

A screenshot of the Anaconda Prompt window. The terminal shows the command '(base) C:\Users\huxiao>conda activate tutorial' and its output '(tutorial) C:\Users\huxiao>'. Then, the user enters the command '(tutorial) C:\Users\huxiao>python -m pip install jupyter ipykernel'.

```
Anaconda Prompt
Using cached argon2_cffi_bindings-26.1.0-cp310-abi3-win_amd64.whl (25 kB)
Using cached beautifulsoup4-4.15.0-py3-none-any.whl (109 kB)
Downloading soupsieve-2.10-py3-none-any.whl (38 kB)
Using cached defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Using cached fqdn-1.5.1-py3-none-any.whl (9.1 kB)
Using cached isoduration-20.11.0-py3-none-any.whl (11 kB)
Using cached arrow-1.4.0-py3-none-any.whl (68 kB)
Using cached jupyterlab_pygments-0.3.0-py3-none-any.whl (15 kB)
Downloading notebook-7.6.3-py3-none-any.whl (5.5 MB)
5.5/5.5 MB 26.0 MB/s 0:00:00
Using cached uri_template-1.3.0-py3-none-any.whl (11 kB)
Installing collected packages: widgetsnbextension, webencodings, webcolors, uri-template, soupsieve, send2trash, rfc3987
-syntax, rfc3986-validator, pywinpty, python-json-logger, prometheus-client, pandocfilters, overrides, mistune, jupyterl
ab_widgets, jupyterlab-pygments, json5, jinja2, fqdn, fastjsonschema, defusedxml, babel, async-lru, tinycss2, terminado,
jupyter-builder, bleach, beautifulsoup4, arrow, argon2-cffi-bindings, jupyter-server-terminals, isoduration, argon2-cff
i, nbformat, ipywidgets, nbclient, jupyter-events, jupyter-console, nbconvert, jupyter-server, notebook-shim, jupyterlab
-server, jupyter-lsp, jupyterlab, notebook, jupyter
Successfully installed argon2-cffi-25.1.0 argon2-cffi-bindings-26.1.0 arrow-1.4.0 async-lru-2.3.0 babel-2.18.0 beautiful
soup4-4.15.0 bleach-6.4.0 defusedxml-0.7.1 fastjsonschema-2.22.2 fqdn-1.5.1 ipywidgets-8.1.9 isoduration-20.11.0 jinja2-
3.1.6 json5-0.15.0 jupyter-1.1.1 jupyter-builder-1.2.3 jupyter-console-6.6.3 jupyter-events-0.12.1 jupyter-lsp-2.3.1 jup
yter-server-2.21.1 jupyter-server-terminals-0.5.4 jupyterlab-4.6.4 jupyterlab-pygments-0.3.0 jupyterlab-server-2.28.1 ju
pyterlab_widgets-3.0.17 mistune-3.3.4 nbclient-0.11.0 nbconvert-7.17.1 nbformat-5.11.1 notebook-7.6.3 notebook-shim-0.2.
4 overrides-7.7.0 pandocfilters-1.5.1 prometheus-client-0.26.0 python-json-logger-4.2.0 pywinpty-3.0.5 rfc3986-validator
-0.1.1 rfc3987-syntax-1.1.0 send2trash-2.1.0 soupsieve-2.10 terminado-0.18.1 tinycss2-1.5.1 uri-template-1.3.0 webcolors
-25.10.0 webencodings-0.6.1 widgetsnbextension-4.0.16

[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(test1111) C:\Users\huxiao>
```

4. Install LlamaIndex and Other Dependencies

Open the `02_RAG_tutorial` folder.

You can right-click an empty area inside the folder and select:

Open in Terminal

This should open Windows PowerShell in the correct directory.

First, activate your virtual environment:

```
conda activate myenv
```

You should see:

```
(myenv)
```

instead of:

```
(base)
```

Then install all the required packages using:

```
pip install -r requirements.txt
```

The required packages are listed in `requirements.txt`.

The installation may take several minutes, depending on your computer and network connection.

When the command prompt becomes available again, the installation is complete.

Note: You may see warnings during installation. Some warnings are harmless, but if you encounter an actual installation error that prevents the command from completing, please keep the error message use AI to try to fix the problem.

```
Windows PowerShell
Requirement already satisfied: mpmath<1.4,>=1.1.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from sympy>=1.13.3->torch>=2.2->sentence-transformers>=2.2.0->-r requirements.txt (line 30)) (1.3.0)
Requirement already satisfied: shellingham>=1.3.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from typer>=0.9.0->chromadb>=0.4.0->-r requirements.txt (line 24)) (1.5.4)
Requirement already satisfied: mypy-extensions>=0.3.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from typing-inspect>=0.8.0->llama-index-core<0.15.0,>=0.14.24->llama-index>=0.10.0->-r requirements.txt (line 9)) (1.1.0)
Requirement already satisfied: watchfiles>=0.20 in D:\anaconda3\envs\tutorial\Lib\site-packages (from uvicorn[standard]>=0.24.0->-r requirements.txt (line 38)) (1.2.0)
Requirement already satisfied: cffi>=1.0.1 in D:\anaconda3\envs\tutorial\Lib\site-packages (from argon2-cffi-bindings->argon2-cffi>=21.1->jupyter-server<3,>=2.19.0->jupyterlab>=4.0.0->-r requirements.txt (line 80)) (2.1.1)
Requirement already satisfied: pycparser in D:\anaconda3\envs\tutorial\Lib\site-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi>=21.1->jupyter-server<3,>=2.19.0->jupyterlab>=4.0.0->-r requirements.txt (line 80)) (3.0)
Requirement already satisfied: marshmallow<4.0.0,>=3.18.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from dataclasses-json->llama-index-core<0.15.0,>=0.14.24->llama-index>=0.10.0->-r requirements.txt (line 9)) (3.26.2)
Requirement already satisfied: griffecli==2.3.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from griffe->banks<3,>=2.3.0->llama-index-core<0.15.0,>=0.14.24->llama-index>=0.10.0->-r requirements.txt (line 9)) (2.3.0)
Requirement already satisfied: griffe==2.3.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from griffe->banks<3,>=2.3.0->llama-index-core<0.15.0,>=0.14.24->llama-index>=0.10.0->-r requirements.txt (line 9)) (2.3.0)
Requirement already satisfied: widgetsnbextension~4.0.16 in D:\anaconda3\envs\tutorial\Lib\site-packages (from ipywidgets->jupyter>=1.0.0->-r requirements.txt (line 79)) (4.0.16)
Requirement already satisfied: jupyterlab_widgets~3.0.17 in D:\anaconda3\envs\tutorial\Lib\site-packages (from ipywidgets->jupyter>=1.0.0->-r requirements.txt (line 79)) (3.0.17)
Requirement already satisfied: lz4>=3.1.10 in D:\anaconda3\envs\tutorial\Lib\site-packages (from ir-datasets->flagembedding>=1.2.0->-r requirements.txt (line 31)) (4.4.5)
Requirement already satisfied: arrow>=0.15.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.11.0->jupyter-server<3,>=2.19.0->jupyterlab>=4.0.0->-r requirements.txt (line 80)) (1.4.0)
Requirement already satisfied: oauthlib>=3.0.0 in D:\anaconda3\envs\tutorial\Lib\site-packages (from requests-oauthlib->kubernetes>=28.1.0->chromadb>=0.4.0->-r requirements.txt (line 24)) (3.3.1)
(tutorial) PS D:\RAG\02_RAG tutorial>
```

5. Open the Tutorial in Jupyter Notebook

Make sure you are still in the `02_RAG_tutorial` folder.

If necessary, activate your environment again:

```
conda activate myenv
```

Then run:

```
jupyter notebook
```

A Jupyter Notebook window should open in your browser.

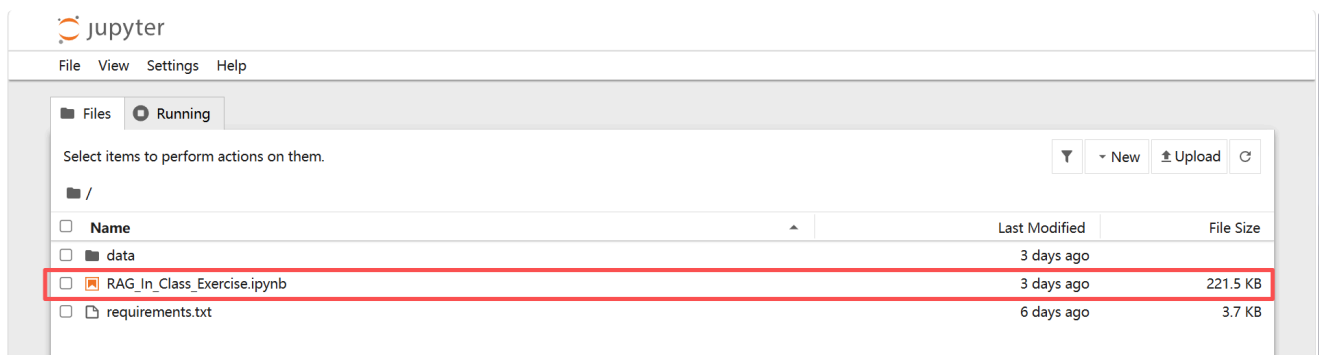
```
npm prefix
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

(base) PS D:\RAG\01_tutorial_KG> conda activate tutorial
(tutorial) PS D:\RAG\01_tutorial_KG> jupyter notebook
[I 2026-09-28 00:02:41.372 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2026-09-28 00:02:41.375 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2026-09-28 00:02:41.379 ServerApp] jupyterlab | extension was successfully linked.
[I 2026-09-28 00:02:41.379 ServerApp] jupyterlab_jupyter_text | extension was successfully linked.
[I 2026-09-28 00:02:41.383 ServerApp] notebook | extension was successfully linked.
[I 2026-09-28 00:02:41.427 ServerApp] notebook_shim | extension was successfully linked.
[I 2026-09-28 00:02:41.475 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-09-28 00:02:41.477 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2026-09-28 00:02:41.477 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2026-09-28 00:02:41.480 LabApp] JupyterLab extension loaded from D:\anaconda3\envs\tutorial\Lib\site-packages\jupyter
lab
[I 2026-09-28 00:02:41.480 LabApp] JupyterLab application directory is D:\anaconda3\envs\tutorial\share\jupyter\lab
[I 2026-09-28 00:02:41.481 LabApp] Extension Manager is 'pypi'.
[I 2026-09-28 00:02:41.511 ServerApp] jupyterlab | extension was successfully loaded.
[I 2026-09-28 00:02:41.511 ServerApp] [JupyterText Server Extension] Deriving an AsyncTextFileContentsManager from AsyncLar
geFileManager
[I 2026-09-28 00:02:41.512 ServerApp] jupyterlab_jupyter_text | extension was successfully loaded.
[I 2026-09-28 00:02:41.516 ServerApp] notebook | extension was successfully loaded.
[I 2026-09-28 00:02:41.517 ServerApp] Serving notebooks from local directory: D:\RAG\01_tutorial_KG
[I 2026-09-28 00:02:41.517 ServerApp] Jupyter Server 2.21.1 is running at:
[I 2026-09-28 00:02:41.517 ServerApp] http://localhost:8888/tree?token=04fbcfa099eb8b8a287e3f85a495d46fa7e324f1fa20c6a2
6a2
[I 2026-09-28 00:02:41.517 ServerApp] http://127.0.0.1:8888/tree?token=04fbcfa099eb8b8a287e3f85a495d46fa7e324f1fa20c
6a2
[I 2026-09-28 00:02:41.517 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
```

Make sure that you launch Jupyter Notebook **after activating your virtual environment**, because the required packages have been installed in this environment.

Open:

RAG_In_Class_Exercise.ipynb



6. Basic Jupyter Notebook Operations

The tutorial uses two types of cells:

- **Markdown cells** — contain explanations and instructions.
- **Code cells** — contain executable Python code.

The tutorial is designed to be followed sequentially.

Please do not randomly skip between cells, because later cells may depend on variables or results generated by earlier cells.

Useful shortcuts:

Shortcut	Action
Shift + Enter	Run the current cell and move to the next cell
Ctrl + Enter	Run the current cell and stay on the current cell

You can also use the toolbar:

- ▶ Run the current cell
- ■ Stop execution
- ↺ Restart the kernel

Restarting the kernel clears the variables stored in memory and resets the Python runtime.

jupyter RAG_In_Class_Exercise Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python 3 (ipykernel)

1.1 Dependency installation

This notebook does not install packages.

Prepare the environment once before class:

```
conda activate myenv #my env 部分请及时替换为你的虚拟环境的名字
pip install -r requirements.txt
```

Then select the **Python (rag-course)** kernel in Jupyter.

Keeping installation outside the notebook avoids package-version changes, kernel mismatch, and unexpected restarts during class. The notebook only checks that the required packages can be imported.

```
[2]: # 检查核心 package
import importlib

packages = {
    "llama_index": "LlamaIndex",
    "chromadb": "Chroma",
    "openai": "OpenAI SDK",
    "pypdf": "PyPDF",
    "dotenv": "python-dotenv",
    "pandas": "Pandas",
    "sklearn": "scikit-learn",
}

print("=" * 60)
print("Package Check")
print("=" * 60)

for module, name in packages.items():
    try:
        importlib.import_module(module)
        print(f"✅ {name}")
    except ImportError as e:
        print(f"❌ {name}: {e}")
```

```
=====
Package Check
=====
✅ LlamaIndex
✅ Chroma
✅ OpenAI SDK
✅ PyPDF
✅ python-dotenv
✅ Pandas
✅ scikit-learn
```

Before Coming to Class

Please make sure that you have completed the following:

Knowledge Graph

- Downloaded the `01_tutorial_KG` folder
- Looked through the two datasets
- Created a Neo4j account
- Created a free Neo4j Aura instance
- Imported the `dataset01_in-class-exercise` CSV files

RAG

- Downloaded the `02_RAG_tutorial` folder
- Created an Alibaba Cloud Model Studio / Bailian account
- Obtained an API Key
- Added your API Key and Base URL to `.env`
- Installed Miniconda
- Created and activated a Python 3.10 environment
- Installed Jupyter Notebook and the required dependencies
- Successfully opened `RAG_In_Class_Exercise.ipynb`

You **do not need to complete the tutorial exercises before class**. The purpose of this preparation is to make sure that your accounts, datasets, and software environment are ready.

If everything above is ready, you are all set for the tutorial.

See you in class!